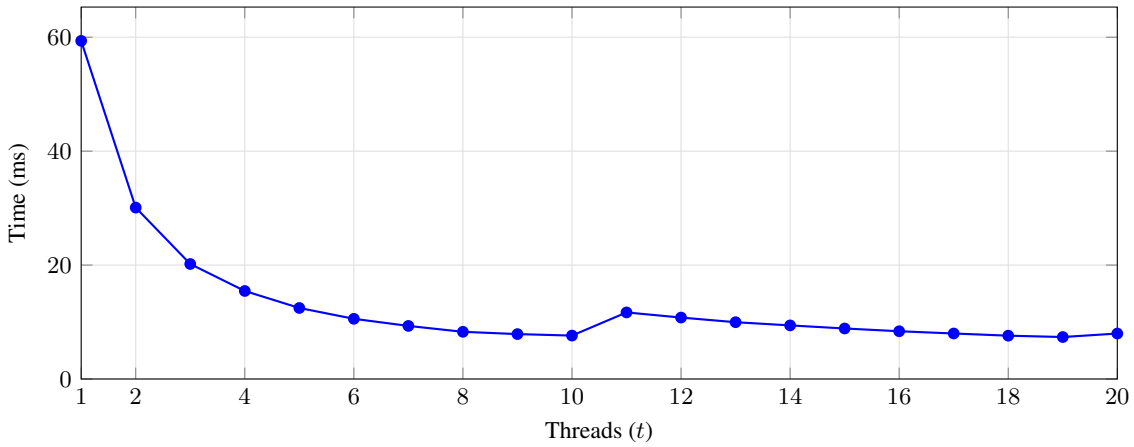
ASSIGNMENT 3

Sammarth Kapse
9088922829
https://github.com/sammarth-kapse/repo759/tree/main/HW03

All measurements were collected on Euler node `euler01.engr.wisc.edu` in Slurm job 91407 with 20 CPUs requested. Each point is the median of 3 runs; timings cover the kernel function only.

Task 1

Matrix multiplication retains the i–k–j loop order from HW02. An OpenMP parallel loop distributes rows of C among threads. Each thread initializes and computes its own rows, so no reduction or lock is needed.
For $n = 1024$, the median time changed from 59.357 ms at one thread to 7.9813 ms at 20 threads, a speedup of $7.437\times$. The fastest measured configuration used 19 threads (7.3682 ms; $8.0558\times$ speedup).



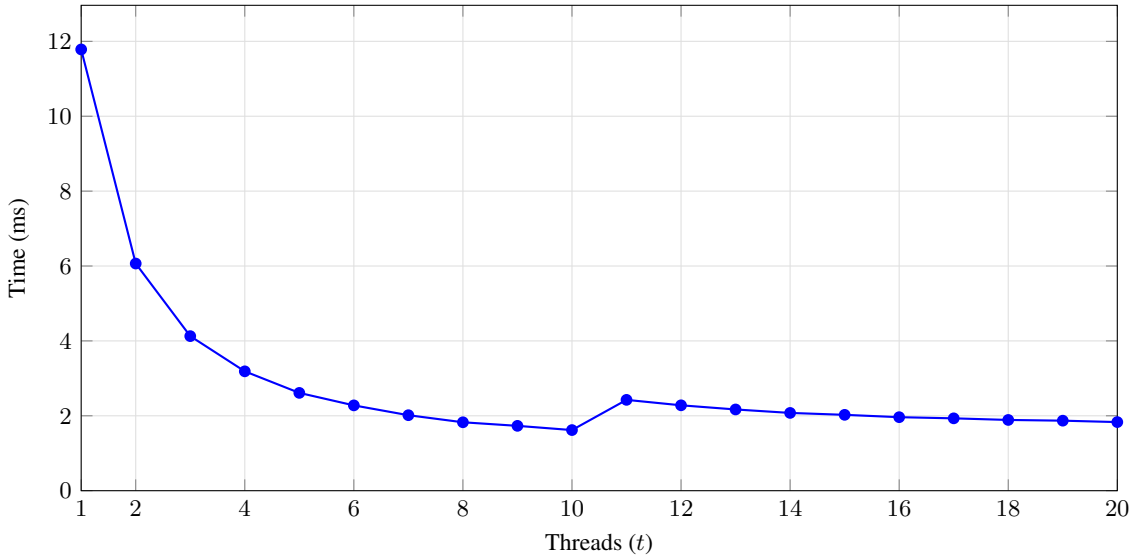
Matrix multiplication: $n = 1024$.

Task 2

An OpenMP parallel loop distributes output rows among threads. The input image and mask are read-only, and each output pixel has a private accumulator. The HW02 boundary convention is preserved: ones on edge strips and zeros where both coordinates are outside the image.

For $n = 1024$ and a 3×3 mask, the time at one thread was 11.784 ms. The fastest median was 1.6188 ms at 10 threads (7.279× speedup). At 20 threads the time was 1.8323 ms, 13.187% above the minimum; using all 20 threads did not improve the measured minimum. Increasing from 10 to 20 threads changed the time from 1.6188 to 1.8323 ms, so the additional threads did not reduce runtime in that comparison.

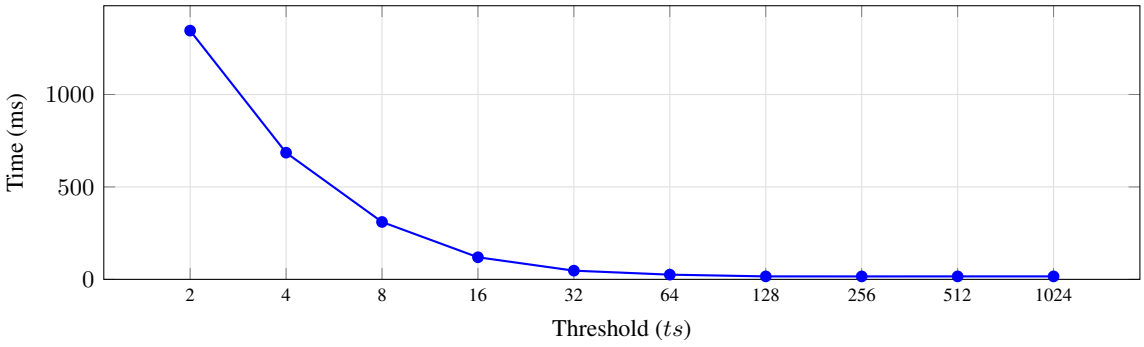
A plausible explanation for diminishing returns is that each pixel requires little work: OpenMP team and scheduling costs become substantial, while threads share caches and memory bandwidth. These are performance interpretations; the timing experiment does not measure those costs separately.



Convolution: $n = 1024$, mask size 3×3.

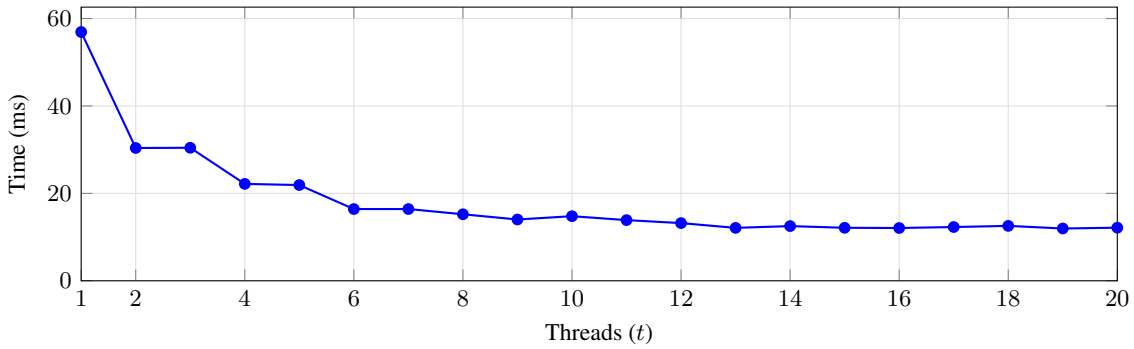
Task 3

Merge sort creates tasks for the two halves, waits for both, and then merges their sorted results. Below the threshold, iterative heap sorting avoids recursive calls and task creation.
For $n = 10^6$ and eight threads, the best tested threshold was $ts = 256$, with a median time of 15.926 ms. The smallest threshold, $ts = 2$, took 1345.5 ms. Small thresholds create many tiny tasks, increasing scheduling overhead. Large thresholds leave more work in serial leaf sorts; the sweep measures this tradeoff for the tested implementation and node.



Threshold sweep: $n = 10^6$, $t = 8$; logarithmic horizontal axis.

At $ts = 256$, the fastest thread count was 19 (11.963 ms). One thread took 56.923 ms and 20 threads took 12.149 ms ($4.6854\times$ speedup). Serial merging near the top of the recursion tree and task overhead limit ideal scaling.



Thread sweep: $n = 10^6$, selected threshold $ts = 256$.